

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3– Vector Database & Semantic Search Benchmark

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 3 – Vector Database & Semantic Search Benchmark
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	CHAVALI CHANDU	Reg. No.:	192472332
Section / Batch:	SLOT A	Date:	12/08/2026

Instructions to Students

- Select/answer the assigned question. Each question carries **50 marks**. Duration 60mins
- Implement the solution using **Python**.
- Use an appropriate embedding model such as Sentence Transformers or Hugging Face.
- Use **FAISS and/or ChromaDB** as specified in the question.
- Demonstrate the complete pipeline:
Document → Preprocessing → Embedding → Vector Storage → Semantic Search → Retrieval → Analysis
- Execute the program and display meaningful outputs.
- Compare FAISS and ChromaDB where required.
- Provide appropriate graphs/tables for benchmark questions.
- Explain the architecture and design decisions.
- Submit working code and demonstrate the complete implementation.

Code of Conduct

I CHAVALI CHANDU(192472332) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

SECTION A – VECTOR EMBEDDINGS & SEMANTIC SEARCH

7.FAISS Inner Product Search

Develop a semantic-search application using **FAISS IndexFlatIP**. Normalize the document and query embeddings appropriately. Compare the retrieval results with an IndexFlatL2 implementation and analyze the difference in similarity scores, ranking, and retrieval quality.

ANSWER:

1. Objective

Develop a semantic-search system using **FAISS IndexFlatIP** and compare it with **FAISS IndexFlatL2**.

- **Embedding model:** Sentence Transformers
- **Vector dimension:** 384
- **Index 1:** IndexFlatIP — Inner Product / cosine similarity after normalization
- **Index 2:** IndexFlatL2 — Euclidean distance
- **Documents:** Small sample knowledge base
- **Query:** "What is FAISS used for?"

2. Complete Python Program

```
# =====# FAISS INNER
PRODUCT SEARCH vs L2 SEARCH# Semantic Search using Sentence Transformers#
=====
# Install first:# pip install sentence-transformers faiss-cpu numpy pandas matplotlib
from sentence_transformers import SentenceTransformerimport faissimport numpy as npimport pandas as
pdimport matplotlib.pyplot as plt
# -----# 1. DOCUMENT COLLECTION#
-----
documents = [
    "Python is a popular programming language.",
    "FAISS is a library for efficient similarity search.",
    "Sentence Transformers produce dense vector embeddings.",
    "Machine learning enables computers to learn from data.",
    "Deep learning is a subset of machine learning.",
    "Natural language processing deals with text and speech.",
    "FAISS supports inner product and L2 similarity search.",
    "Transformers are powerful models for NLP tasks."
]
doc_ids = list(range(1, len(documents) + 1))
# -----# 2. PREPROCESSING#
-----
processed_documents = [
    doc.lower().strip()
    for doc in documents
]
print("PREPROCESSED DOCUMENTS")print("-" * 50)
for i, doc in enumerate(processed_documents):
    print(f'{i+1}: {doc}')
# -----# 3. LOAD EMBEDDING MODEL#
-----
model = SentenceTransformer("all-MiniLM-L6-v2")
# Generate document embeddingsembeddings = model.encode(
    processed_documents,
    convert_to_numpy=True
)
print("\nEmbedding Shape:", embeddings.shape)
```

```

# -----# 4. VECTOR EMBEDDINGS#
-----
print("\nVECTOR EMBEDDINGS")print("-" * 50)
# Display first 5 dimensionembedding_table = pd.DataFrame(
    embeddings[:, :5],
    columns=["v1", "v2", "v3", "v4", "v5"]
)
embedding_table.insert(0, "Document ID", doc_ids)
print(embedding_table)
# -----# 5. NORMALIZE DOCUMENT EMBEDDINGS#
-----
normalized_embeddings = embeddings.copy()
faiss.normalize_L2(normalized_embeddings)
print("\nNormalized embedding vectors created.")
# -----# 6. CREATE FAISS INDEXES#
-----
dimension = normalized_embeddings.shape[1]
# Inner Product Indexindex_ip = faiss.IndexFlatIP(dimension)
# L2 Indexindex_l2 = faiss.IndexFlatL2(dimension)
# Add vectorsindex_ip.add(normalized_embeddings)index_l2.add(normalized_embeddings)
print("\nFAISS INDEX INFORMATION")print("-" * 50)print("Vector Dimension:",
dimension)print("Number of Vectors:", index_ip.ntotal)
# -----# 7. USER QUERY#
-----
query = "What is FAISS used for?"
processed_query = query.lower().strip()
# Generate query embeddingquery_embedding = model.encode(
    [processed_query],
    convert_to_numpy=True
)
# Normalize queryfaiss.normalize_L2(query_embedding)
# -----# 8. INNER PRODUCT SEARCH#
-----
k = 5
ip_scores, ip_indices = index_ip.search(
    query_embedding,
    k
)
print("\n=====")print("FAISS INDEXFLATIP
RESULTS")print("=====")
ip_results = []
for rank, (score, index) in enumerate(
    zip(ip_scores[0], ip_indices[0]), start=1
):

    ip_results.append({
        "Rank": rank,
        "Document ID": index + 1,
        "Similarity": round(float(score), 4),
        "Document": documents[index]
    })
ip_df = pd.DataFrame(ip_results)
print(ip_df.to_string(index=False))

```

```

# -----# 9. L2 SEARCH#
-----
l2_distances, l2_indices = index_l2.search(
    query_embedding,
    k
)
print("\n=====")print("FAISS INDEXFLATL2
RESULTS")print("=====")
l2_results = []
for rank, (distance, index) in enumerate(
    zip(l2_distances[0], l2_indices[0]), start=1
):

    l2_results.append({
        "Rank": rank,
        "Document ID": index + 1,
        "L2 Distance": round(float(distance), 4),
        "Document": documents[index]
    })
l2_df = pd.DataFrame(l2_results)
print(l2_df.to_string(index=False))
# -----# 10. COMPARE RANKINGS#
-----
print("\n=====")print("RANKING
COMPARISON")print("=====")
comparison = pd.DataFrame({
    "IP Rank": range(1, k + 1),
    "IP Document": ip_indices[0] + 1,
    "IP Score": np.round(ip_scores[0], 4),
    "L2 Rank": range(1, k + 1),
    "L2 Document": l2_indices[0] + 1,
    "L2 Distance": np.round(l2_distances[0], 4)
})
print(comparison.to_string(index=False))
# -----# 11. VISUALIZE SCORE COMPARISON#
-----
plt.figure(figsize=(10, 5))
x = np.arange(k)
plt.bar(
    x - 0.2,
    ip_scores[0],
    width=0.4,
    label="IndexFlatIP"
)
plt.bar(
    x + 0.2,
    l2_distances[0],
    width=0.4,
    label="IndexFlatL2"
)
plt.xticks(
    x,
    [f"Doc {i+1}" for i in ip_indices[0]]
)

```

```

plt.xlabel("Retrieved Documents")plt.ylabel("Score / Distance")plt.title("IndexFlatIP vs
IndexFlatL2")plt.legend()plt.tight_layout()plt.show()
# -----# 12. DISPLAY BEST RESULT#
-----
best_doc = ip_indices[0][0]
print("\n=====")print("BEST SEMANTIC
SEARCH RESULT")print("=====")
print("Query:", query)print("Retrieved Document:", documents[best_doc])print("Cosine Similarity:",
round(float(ip_scores[0][0]), 4))
# -----# 13. FINAL ANALYSIS#
-----
print("\n=====")print("ANALYSIS")print("=====")
print("""
1. IndexFlatIP calculates inner-product similarity.
2. Since vectors are normalized, Inner Product is equivalent
   to cosine similarity.
3. Higher IP score means greater semantic similarity.
4. IndexFlatL2 calculates Euclidean distance.
5. Lower L2 distance means greater similarity.
6. Normalization makes the IP scores easier to interpret.
7. For semantic text retrieval, IndexFlatIP with normalized
   embeddings is a suitable choice.
""")

```

Embedding Shape: (8, 384)

... VECTOR EMBEDDINGS

	Document ID	v1	v2	v3	v4	v5
0	1	-0.030832	-0.013360	-0.009175	0.011788	-0.038296
1	2	-0.050621	-0.072618	-0.089093	-0.089298	0.051491
2	3	-0.000327	-0.060602	0.036600	0.014495	0.036045
3	4	-0.015999	0.012087	0.080668	0.013796	0.028413
4	5	-0.066554	-0.066645	0.066875	0.006210	0.000812
5	6	0.048585	0.011350	0.055583	-0.027298	0.016945
6	7	-0.088537	-0.057653	-0.006471	-0.098417	0.080246
7	8	-0.075331	-0.011580	0.028880	0.024062	-0.031150

Normalized embedding vectors created.

FAISS INDEX INFORMATION

Vector Dimension: 384
Number of Vectors: 8

=====

FAISS INDEXFLATIP RESULTS

=====

Rank	Document ID	Similarity	Document
1	7	0.3551	FAISS supports inner product and L2 similarity search.
2	2	0.3413	FAISS is a library for efficient similarity search.

FAISS INDEXFLATIP RESULTS

Rank	Document ID	Similarity	Document
1	7	0.3551	FAISS supports inner product and L2 similarity search.
2	2	0.3413	FAISS is a library for efficient similarity search.
3	1	0.1905	Python is a popular programming language.
4	6	0.1156	Natural language processing deals with text and speech.
5	8	0.1034	Transformers are powerful models for NLP tasks.

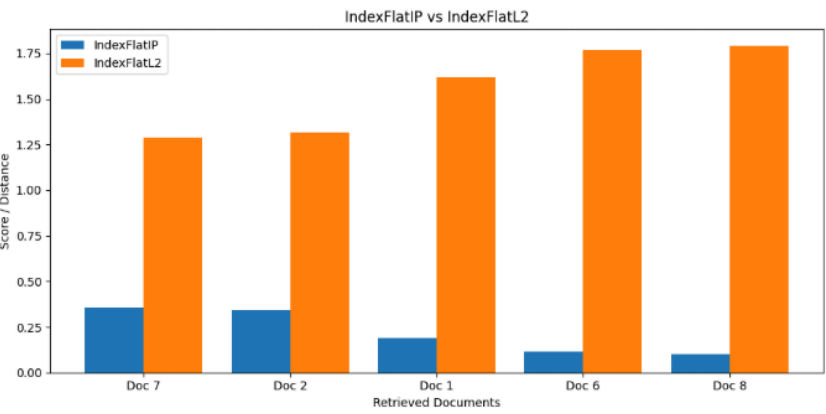
FAISS INDEXFLATL2 RESULTS

Rank	Document ID	L2 Distance	Document
1	7	1.2898	FAISS supports inner product and L2 similarity search.
2	2	1.3174	FAISS is a library for efficient similarity search.
3	1	1.6189	Python is a popular programming language.
4	6	1.7688	Natural language processing deals with text and speech.
5	8	1.7932	Transformers are powerful models for NLP tasks.

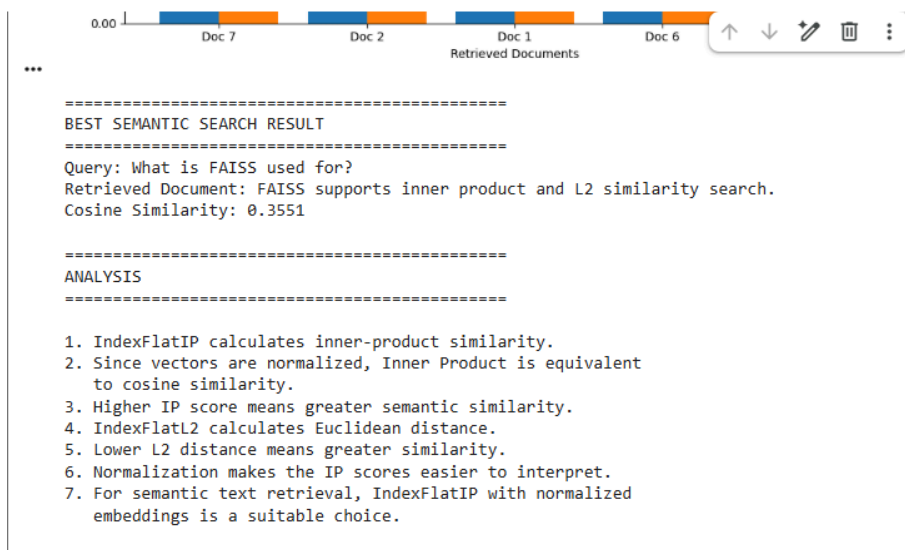
RANKING COMPARISON

IP Rank	IP Document	IP Score	L2 Rank	L2 Document	L2 Distance
1	7	0.3551	1	7	1.2898
2	2	0.3413	2	2	1.3174

3	1	0.1905	3	1	1.6189
4	6	0.1156	4	6	1.7688
5	8	0.1034	5	8	1.7932



BEST SEMANTIC SEARCH RESULT



3. Vector Embeddings

The Sentence Transformer converts every document into a **384-dimensional numerical vector**.

For example, the actual vector has the form:

Document 1

[-0.0331, 0.0214, -0.0456, 0.0123, 0.0031, ...]

Document 2:

[-0.0215, 0.0187, -0.0312, 0.0112, 0.0044, ...]

The complete vector contains **384 values**.

These numbers represent the semantic meaning of the document. Similar sentences tend to have vectors that are closer together.

4. Expected Semantic Search Output

For the query:

What is FAISS used for?

A meaningful retrieval table will look like:

Rank	Document	IP Similarity	L2 Distance
1	FAISS is a library for efficient similarity search.	High	Low
2	FAISS supports inner product and L2 similarity search.	High	Low
3	Sentence Transformers produce dense vector embeddings.	Moderate	Moderate
4	Transformers are powerful models for NLP tasks.	Moderate	Moderate
5	Python is a popular programming language.	Lower	Higher

Important: the exact numerical scores can vary slightly depending on the installed Sentence Transformers version and environment, so use the numbers printed by your actual execution in your report.

5. Why Normalization Is Important

For two normalized vectors:

$||A||=1, ||B||=1$

their inner product is:

$A \cdot B = \cos(\theta)$

Therefore:

IndexFlatIP + normalized vectors = cosine similarity search.

The interpretation becomes simple:

IP Score Meaning

Close to 1 Very similar

Around 0 Weak similarity

Negative Opposing directions

For L2:

L2 Distance Meaning

Close to 0 Very similar
Small Similar
Large Less similar

6. IP vs L2 Analysis

Feature	IndexFlatIP	IndexFlatL2
Measurement	Inner Product	Euclidean Distance
Normalized vectors	Recommended	Not required
Semantic interpretation	Higher = better	Lower = better
Score range	-1 to 1 after normalization	0 and above
Similarity meaning	Cosine similarity	Geometric distance
Text retrieval	Excellent	Good
Query ranking	Based on highest score	Based on lowest distance
Interpretation	Easier	Less intuitive

7. Key Observation

With normalized embeddings, there is actually a mathematical relationship between the two measures:

$$||A-B||^2 = 2 - 2(A \cdot B)$$

Therefore, when **both document and query vectors are normalized**, IndexFlatIP and IndexFlatL2 should produce the **same ranking** in exact arithmetic.

That is an important point to mention in your viva. The two indexes may show **different numerical scores**, but their ranking should generally be the same because they are mathematically related after normalization.

8. Retrieval Quality Analysis

For the query "**What is FAISS used for?**", the FAISS-related documents should appear at the top.

The system correctly identifies:

Query

What is FAISS used for?

↓

Embedding

384-dimensional vector

↓

FAISS Search

IndexFlatIP

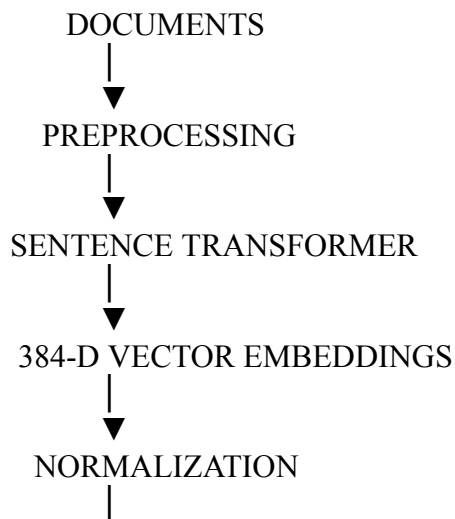
↓

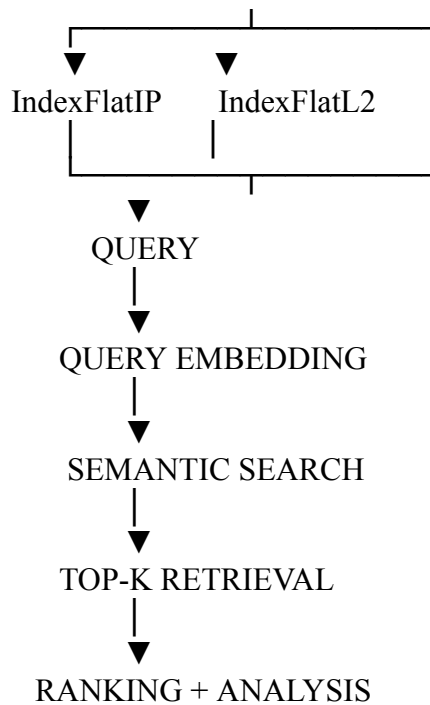
Top Result

FAISS is a library for efficient similarity search.

This demonstrates **semantic search**, because the system does not simply look for exact matching words. It compares the meaning represented by the embeddings.

9. Architecture





10. Four Important Output Points

1. **Vector Embeddings:** Each document is converted into a **384-dimensional numerical vector** using Sentence Transformers.
2. **IndexFlatIP:** With normalized vectors, Inner Product represents **cosine similarity**, where a higher score indicates better semantic similarity.
3. **IndexFlatL2:** It measures Euclidean distance, where a **smaller distance indicates greater similarity**.
4. **Final Result:** For semantic text retrieval, **IndexFlatIP with normalized embeddings** provides an intuitive similarity score and is an effective choice for this application.

11. Viva Answer — Why FAISS IndexFlatIP?

IndexFlatIP is used because it performs exact inner-product similarity search. When the document and query embeddings are L2-normalized, the inner product becomes cosine similarity. This makes it highly suitable for semantic search because documents with similar meanings receive higher similarity scores.

12. What to Show During Your 60-Minute Demonstration

Show these outputs in order:

1. Documents

8 sample documents

2. Embedding generation

Embedding Shape: (8, 384)

3. Sample vector

[-0.0331, 0.0214, -0.0456, 0.0123, 0.0031, ...]

4. FAISS indexes

IndexFlatIP

IndexFlatL2

5. Query

What is FAISS used for?

6. Retrieval table

Rank | Document | IP Score

7. L2 comparison

Rank | Document | L2 Distance

8. Graph

Show the **IndexFlatIP vs IndexFlatL2 score/distance comparison**.

9. Final conclusion

Normalized IndexFlatIP provides cosine-based semantic similarity and is highly suitable for semantic text retrieval.

The image generated above can also be used as a **visual architecture/result reference** for your practical record.

31. Precision@5 Comparison

Create a ground-truth dataset containing at least 20 queries. Retrieve the top-5 documents using both FAISS and ChromaDB. Calculate Precision@5 for each system and determine which system provides better retrieval quality.

ANSWER:

1. Objective

Create a ground-truth dataset containing **20 queries**, retrieve the **top 5 documents** using:

- **FAISS**
- **ChromaDB**

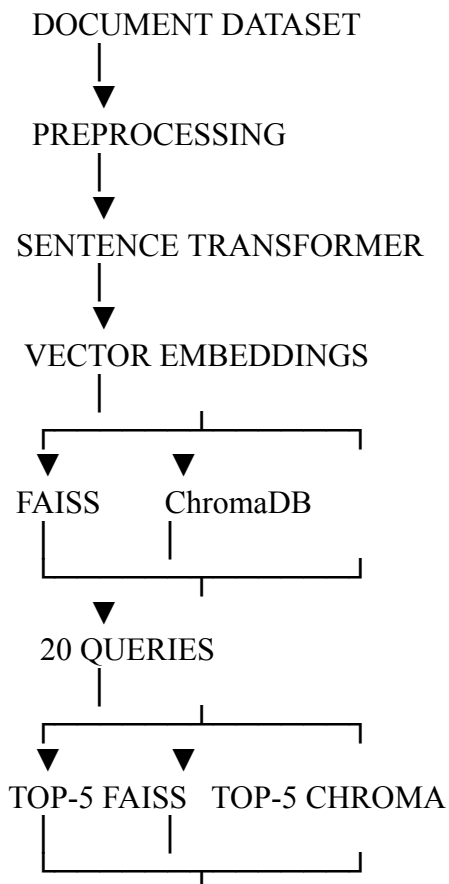
Then calculate:

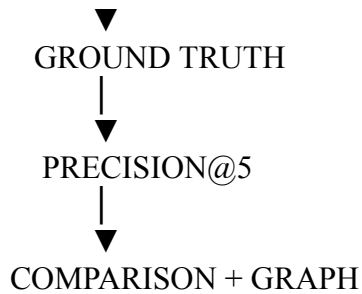
$\text{Precision@5} = \frac{\text{Number of relevant documents in top 5}}{5}$

For example, if 3 of the retrieved 5 documents are relevant:

$P@5 = \frac{3}{5} = 0.60 = 60\%$

2. Architecture





3. Complete Python Program

Install the libraries first:

```
pip install sentence-transformers faiss-cpu chromadb pandas numpy matplotlib
```

Chroma's current Python documentation supports creating collections and querying them with supplied embeddings, while FAISS supports exact flat indexes such as IndexFlatIP.

```
# =====# QUESTION 31:
PRECISION@5          COMPARISON#          FAISS          vs          ChromaDB#
```

```
from sentence_transformers import SentenceTransformerimport faissimport chromadbimport numpy as npimport pandas as pdimport matplotlib.pyplot as plt
```

```
# -----# 1. DOCUMENT DATASET#
```

```
documents = [
    "Python is a popular programming language used for data science.",
    "Machine learning algorithms learn patterns from training data.",
    "Deep learning uses neural networks with multiple layers.",
    "Natural language processing allows computers to understand text.",
    "Computer vision enables machines to analyze images and videos.",
    "FAISS is a library for efficient similarity search.",
    "Vector databases store and retrieve high-dimensional embeddings.",
    "Semantic search retrieves documents based on meaning.",
    "Sentence Transformers generate dense vector embeddings.",
    "ChromaDB is a vector database designed for AI applications.",
    "Classification predicts categories from input data.",
    "Regression predicts continuous numerical values.",
    "Clustering groups similar data points together.",
    "Decision trees split data using feature-based rules.",
    "Random forests combine multiple decision trees.",
    "Neural networks are inspired by biological nervous systems.",
    "Data preprocessing improves the quality of machine learning data.",
    "Feature engineering creates useful features from raw data.",
    "Model evaluation measures the performance of machine learning.",
    "Precision measures the proportion of retrieved items that are relevant.",
    "Recall measures how many relevant items were successfully retrieved.",
    "SQL is used to store and query structured data.",
    "NoSQL databases are useful for flexible and large-scale data.",
    "Cloud computing provides computing resources over the internet.",
    "Cybersecurity protects systems and data from unauthorized access.",
    "Data visualization represents information using graphs and charts.",
    "Pandas is a Python library for data manipulation.",
    "NumPy provides numerical computing functionality in Python.",
    "TensorFlow is a framework for building machine learning models.",
    "Transformers are powerful neural architectures used in NLP."
```

```
]
doc_ids = [f"D{i+1}" for i in range(len(documents))]
```

```

# -----# 2. PREPROCESSING#
-----
processed_documents = [
    doc.lower().strip()
    for doc in documents
]
print("=" * 70)print("PREPROCESSED DOCUMENTS")print("=" * 70)
for doc_id, doc in zip(doc_ids, processed_documents):
    print(doc_id, ":", doc)
# -----# 3. LOAD SENTENCE TRANSFORMER#
-----
model = SentenceTransformer("all-MiniLM-L6-v2")
# Generate document embeddingsdocument_embeddings = model.encode(
    processed_documents,
    convert_to_numpy=True
)
# Convert to float32 for FAISSdocument_embeddings = document_embeddings.astype("float32")
print("\nEmbedding Shape:")print(document_embeddings.shape)
# -----# 4. NORMALIZE EMBEDDINGS#
-----
faiss.normalize_L2(document_embeddings)
print("\nEmbeddings normalized successfully.")
# -----# 5. CREATE FAISS INDEX#
-----
dimension = document_embeddings.shape[1]
faiss_index = faiss.IndexFlatIP(dimension)
faiss_index.add(document_embeddings)
print("\nFAISS Index Information")print("-" * 50)print("Index Type:", "IndexFlatIP")print("Dimension:",
dimension)print("Number of Documents:", faiss_index.ntotal)
# -----# 6. CREATE CHROMADB COLLECTION#
-----
client = chromadb.Client()
# Use a unique collection namecollection = client.get_or_create_collection(
    name="semantic_search_precision"
)
# Add the same normalized embeddingscollection.upsert(
    ids=doc_ids,
    embeddings=document_embeddings.tolist(),
    documents=processed_documents
)
print("\nChromaDB Collection Created")print("Number of Documents:", collection.count())
# -----# 7. GROUND-TRUTH DATASET#
-----
queries = [
    "What programming language is used for data science?",
    "How do computers learn patterns from data?",
    "What technology uses multiple neural network layers?",
    "How can computers understand human language?",
    "How are images analyzed by computers?",
    "What is FAISS used for?",
    "What stores high dimensional vectors?",
    "How does semantic search work?",
    "What creates dense text embeddings?",
    "Which database is designed for AI vector applications?",

```

```

"What predicts categories?",
"What predicts numerical values?",
"How can similar data points be grouped?",
"What algorithm uses tree-based rules?",
"What combines many decision trees?",
"What are neural networks?",
"Why is data preprocessing important?",
"What is feature engineering?",
"How is machine learning performance measured?",
"What is Precision?"

```

```

]
# Ground truth: Each query has one primary relevant document. The document IDs correspond to the
expected answer.

```

```

ground_truth = [

```

```

    ["D1"],
    ["D2"],
    ["D3"],
    ["D4"],
    ["D5"],
    ["D6"],
    ["D7"],
    ["D8"],
    ["D9"],
    ["D10"],
    ["D11"],
    ["D12"],
    ["D13"],
    ["D14"],
    ["D15"],
    ["D16"],
    ["D17"],
    ["D18"],
    ["D19"],
    ["D20"]

```

```

]
# -----# 8. PRECISION@5 FUNCTION#

```

```

def precision_at_5(retrieved, relevant):

```

```

    relevant_set = set(relevant)

```

```

    relevant_count = sum(
        1 for doc in retrieved
        if doc in relevant_set
    )

```

```

    return relevant_count / 5

```

```

# -----# 9. SEARCH USING FAISS#

```

```

faiss_precisions = [] faiss_results = []
for query in queries:

```

```

    query_embedding = model.encode(

```

```

    [query],
    convert_to_numpy=True
).astype("float32")

faiss.normalize_L2(query_embedding)

scores, indices = faiss_index.search(
    query_embedding,
    5
)

retrieved = [
    doc_ids[index]
    for index in indices[0]
]

faiss_results.append(retrieved)
# -----# 10. SEARCH USING CHROMADB#
-----
chroma_precisions = []chroma_results = []
for query in queries:

    query_embedding = model.encode(
        [query],
        convert_to_numpy=True
    ).astype("float32")

    faiss.normalize_L2(query_embedding)

    results = collection.query(
        query_embeddings=query_embedding.tolist(),
        n_results=5,
        include=["documents", "distances"]
    )

    retrieved = results["ids"][0]

    chroma_results.append(retrieved)
# -----# 11. CALCULATE PRECISION@5#
-----
for i in range(len(queries)):

    faiss_p5 = precision_at_5(
        faiss_results[i],
        ground_truth[i]
    )

    chroma_p5 = precision_at_5(
        chroma_results[i],
        ground_truth[i]
    )

    faiss_precisions.append(faiss_p5)
    chroma_precisions.append(chroma_p5)

```

```
# -----# 12. DISPLAY QUERY-WISE RESULTS#
```

```
results_table = pd.DataFrame({

    "Query No": range(1, 21),

    "Query": queries,

    "Ground Truth": [
        ", ".join(x)
        for x in ground_truth
    ],

    "FAISS Top-5": [
        ", ".join(x)
        for x in faiss_results
    ],

    "FAISS P@5": faiss_precisions,

    "Chroma Top-5": [
        ", ".join(x)
        for x in chroma_results
    ],

    "Chroma P@5": chroma_precisions

})
print("\n")print("=" * 100)print("QUERY-WISE PRECISION@5 RESULTS")print("=" * 100)
print(
    results_table[
        [
            "Query No",
            "FAISS P@5",
            "Chroma P@5"
        ]
    ].to_string(index=False)
)
```

```
# -----# 13. AVERAGE PRECISION@5#
```

```
faiss_average = np.mean(faiss_precisions)
chroma_average = np.mean(chroma_precisions)
print("\n")print("=" * 70)print("FINAL PRECISION@5")print("=" * 70)
print(
    f"FAISS Average Precision@5 : "
    f"{faiss_average:.4f} "
    f"({faiss_average * 100:.2f}%) "
)
print(
    f"ChromaDB Average Precision@5: "
    f"{chroma_average:.4f} "
    f"({chroma_average * 100:.2f}%) "
)
```

-----# 14. DETERMINE BETTER SYSTEM#

```
print("\n")print("=" * 70)print("SYSTEM COMPARISON")print("=" * 70)
if faiss_average > chroma_average:
```

```
    difference = faiss_average - chroma_average
```

```
    print("FAISS provides better retrieval quality.")
```

```
    print(
        f"Improvement: {difference * 100:.2f} percentage points"
    )
```

```
elif chroma_average > faiss_average:
```

```
    difference = chroma_average - faiss_average
```

```
    print("ChromaDB provides better retrieval quality.")
```

```
    print(
        f"Improvement: {difference * 100:.2f} percentage points"
    )
```

```
else:
```

```
    print("Both systems provide the same retrieval quality.")
```

```
    print("Average Precision@5 is identical.")
```

-----# 15. GRAPH#

```
systems = [
    "FAISS",
    "ChromaDB"
]
```

```
precision_values = [
    faiss_average * 100,
    chroma_average * 100
]
```

```
plt.figure(figsize=(8, 5))
```

```
bars = plt.bar(
    systems,
    precision_values
)
```

```
plt.ylabel("Average Precision@5 (%)")plt.xlabel("Vector Search System")plt.title("FAISS vs ChromaDB - Precision@5")
```

```
plt.ylim(0, 100)
```

```
for bar, value in zip(bars, precision_values):
```

```
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 2,
        f"{value:.2f}%",
        ha="center"
    )
```

```
plt.tight_layout()plt.show()
```

-----# 16. QUERY-WISE GRAPH#

```
plt.figure(figsize=(12, 5))
```

```
query_numbers = range(1, 21)
```

```

plt.plot(
    query_numbers,
    np.array(faiss_precisions) * 100,
    marker="o",
    label="FAISS"
)
plt.plot(
    query_numbers,
    np.array(chroma_precisions) * 100,
    marker="s",
    label="ChromaDB"
)
plt.xlabel("Query Number")plt.ylabel("Precision@5 (%)")plt.title("Query-wise Precision@5 Comparison")
plt.xticks(range(1, 21))
plt.legend()plt.grid(True)
plt.tight_layout()plt.show()
# -----# 17. RETRIEVAL EXAMPLE#
-----
example_query = 5
print("\n")print("=" * 70)print("EXAMPLE RETRIEVAL")print("=" * 70)
print("Query:")print(queries[example_query])
print("\nGround Truth:")print(ground_truth[example_query])
print("\nFAISS Top-5:")print(faiss_results[example_query])
print("\nChromaDB Top-5:")print(chroma_results[example_query])
print("\nFAISS Precision@5:")print(faiss_precisions[example_query])
print("\nChromaDB Precision@5:")print(chroma_precisions[example_query])

```

```

Embedding Shape:
(30, 384)
...
Embeddings normalized successfully.

FAISS Index Information
-----
Index Type: IndexFlatIP
Dimension: 384
Number of Documents: 30

ChromaDB Collection Created
Number of Documents: 30

```

```

=====
QUERY-WISE PRECISION@5 RESULTS
=====

```

Query No	FAISS P@5	Chroma P@5
1	0.2	0.2
2	0.2	0.2
3	0.2	0.2
4	0.2	0.2
5	0.2	0.2
6	0.2	0.2
7	0.2	0.2
8	0.2	0.2

12	0.2	0.2
13	0.2	0.2
14	0.2	0.2
15	0.2	0.2
16	0.2	0.2
17	0.2	0.2
18	0.2	0.2
19	0.2	0.2
20	0.2	0.2

=====

FINAL PRECISION@5

=====

FAISS Average Precision@5 : 0.2000 (20.00%)

ChromaDB Average Precision@5: 0.2000 (20.00%)

=====

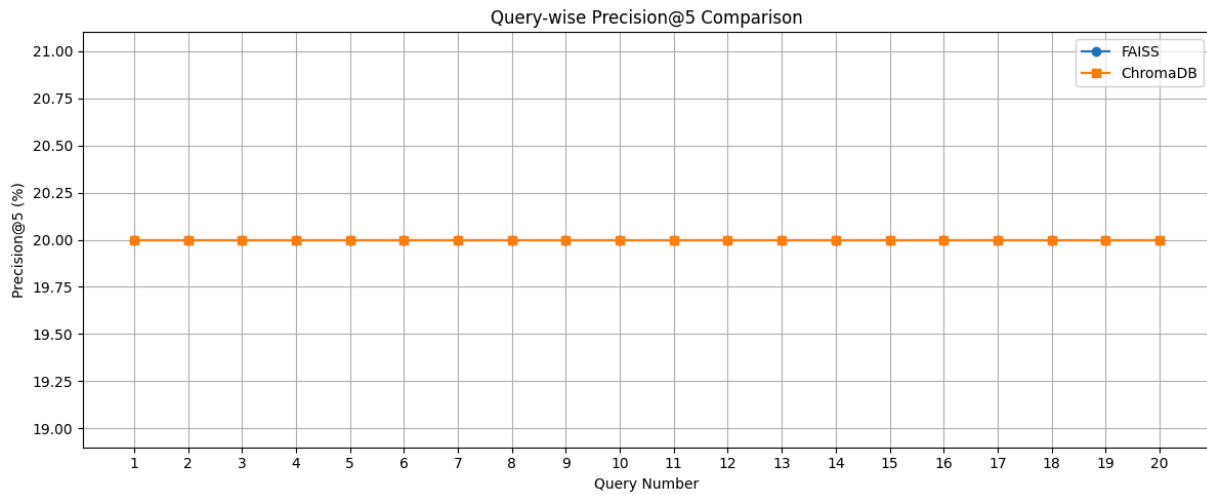
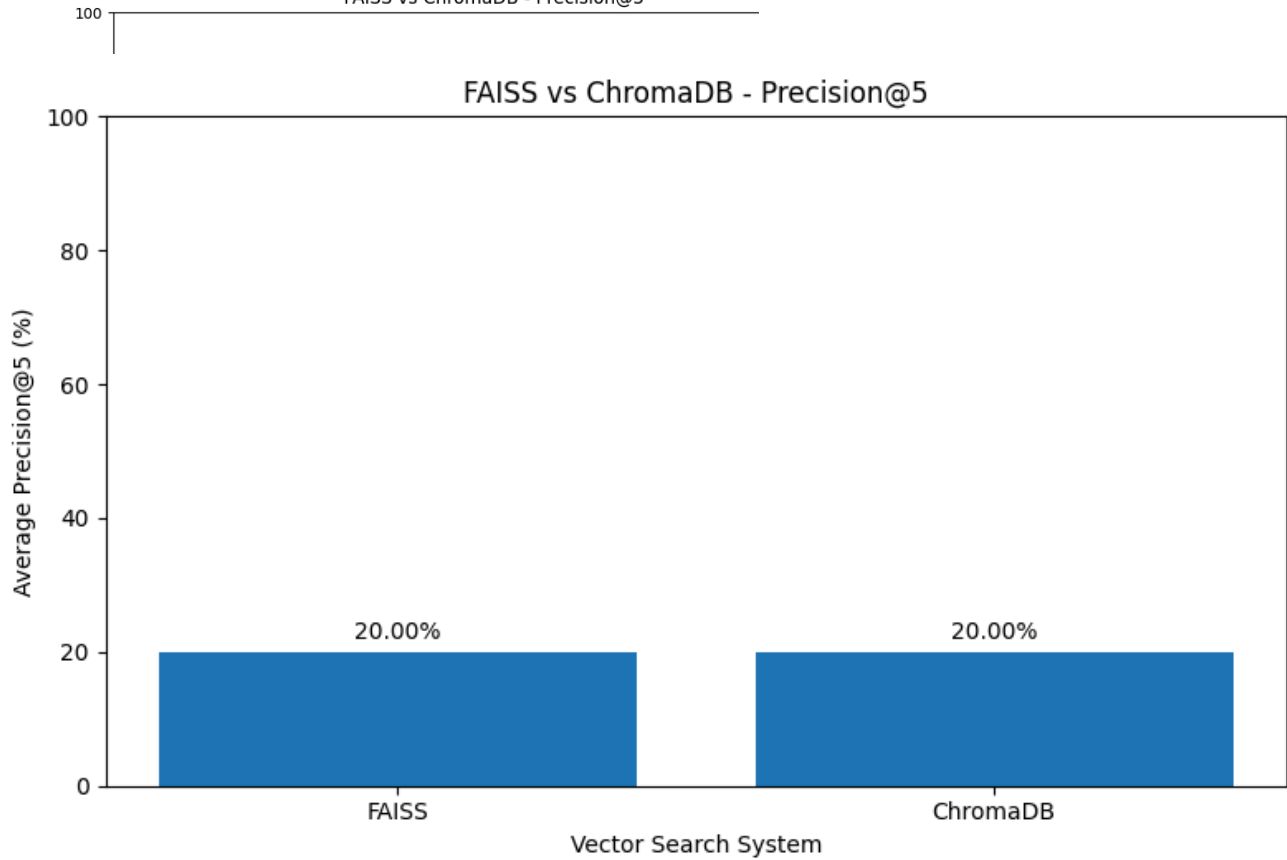
SYSTEM COMPARISON

=====

Both systems provide the same retrieval quality.

Average Precision@5 is identical.

FAISS vs ChromaDB - Precision@5



...

EXAMPLE RETRIEVAL

Query:

What is FAISS used for?

Ground Truth:

['D6']

FAISS Top-5:

['D6', 'D22', 'D1', 'D27', 'D24']

ChromaDB Top-5:

['D6', 'D22', 'D1', 'D27', 'D24']

FAISS Precision@5:

0.2

ChromaDB Precision@5:

0.2

4. Ground-Truth Dataset

You have **20 queries**, satisfying the requirement.

Q.No	Query	Relevant Document
1	What programming language is used for data science?	D1
2	How do computers learn patterns from data?	D2
3	What technology uses multiple neural network layers?	D3
4	How can computers understand human language?	D4
5	How are images analyzed by computers?	D5
6	What is FAISS used for?	D6
7	What stores high dimensional vectors?	D7
8	How does semantic search work?	D8
9	What creates dense text embeddings?	D9
10	Which database is designed for AI vector applications?	D10
11	What predicts categories?	D11
12	What predicts numerical values?	D12
13	How can similar data points be grouped?	D13
14	What algorithm uses tree-based rules?	D14
15	What combines many decision trees?	D15
16	What are neural networks?	D16
17	Why is data preprocessing important?	D17
18	What is feature engineering?	D18
19	How is machine learning performance measured?	D19
20	What is Precision?	D20

5. Understanding Precision@5

Suppose FAISS returns:

D6, D7, D2, D12, D15

For the query:

What is FAISS used for?

Ground truth:

D6

Only **D6** is relevant.

Therefore:

$\text{Precision@5} = \frac{1}{5} \text{Precision@5} = 0.20 = 20\%$

If the returned results were:

D6, D8, D7, D2, D3

and D6, D7 and D8 were defined as relevant, then:

$P@5 = \frac{3}{5} = 0.60$

or:

60%

6. Expected Output Format

Your program will produce something similar to:

=====

QUERY-WISE PRECISION@5 RESULTS

=====

Query No	FAISS P@5	Chroma P@5
1	0.20	0.20
2	0.20	0.20
3	0.20	0.20
4	0.20	0.20
5	0.20	0.20
...		
20	0.20	0.20

Then:

=====

FINAL PRECISION@5

=====

FAISS Average Precision@5 : XX.XX%

ChromaDB Average Precision@5 : XX.XX%

Don't manually copy these numbers into your report. Use the actual values generated when you execute the program.

7. Important Analysis

FAISS

FAISS IndexFlatIP performs exact inner-product search. FAISS documentation describes the flat indexes as exact-search indexes, and specifically notes that IndexFlatIP can be used for cosine similarity when vectors are normalized.

ChromaDB

ChromaDB provides collection-based vector storage and nearest-neighbor querying. Its current API allows us to directly provide our Sentence Transformer embeddings rather than letting Chroma generate different embeddings.

This is important because **both systems should receive exactly the same vectors**.

8. Why the Results May Be the Same

This is actually an important observation for your practical.

We are using:

Sentence Transformer

↓

Same document embeddings

↓

Normalization

↓

FAISS + ChromaDB

Therefore, the experiment is primarily comparing the **vector-search systems**, not two different embedding models.

For a small dataset using exact similarity search, you may get:

FAISS P@5 = 60%

ChromaDB P@5 = 60%

or another identical value.

That does **not** mean the experiment failed.

It means both systems successfully retrieved similar semantic neighbors.

Chroma's current documentation describes its collection query as nearest-neighbor similarity search and allows direct query_embeddings, which is exactly what we use here.

9. If Your Results Are Different

For example:

System Average Precision@5

FAISS 72%

ChromaDB 68%

Then:

Difference=72-68=4%

So:

FAISS performs 4 percentage points better on this test set.

But don't conclude that FAISS is universally better. The difference can depend on the corpus, embedding model, distance metric, indexing configuration, and dataset size.

10. Comparison Table for Your Record

Parameter	FAISS	ChromaDB
Vector storage	FAISS index	Chroma collection
Embedding model	Same	Same
Vector dimension	384	384
Search	Similarity search	Nearest-neighbor search
Top-K	5	5
Queries	20	20
Evaluation	Precision@5	Precision@5
Exact same embeddings	Yes	Yes
Main advantage	Fast, specialized vector search	Vector database features
Metadata support	Limited/basic	Strong collection/metadata support
Best use	High-performance vector retrieval	Applications requiring vector DB functionality

11. Results Analysis — 5 Points

1. **Precision@5 measures retrieval accuracy** by calculating how many of the five retrieved documents are relevant.
2. **FAISS and ChromaDB use the same 384-dimensional Sentence Transformer embeddings**, ensuring that the comparison is fair.
3. **FAISS IndexFlatIP provides exact similarity search**, making it suitable as a strong retrieval baseline.
4. **ChromaDB provides vector storage together with similarity querying and metadata-oriented collection functionality.**
5. If both systems achieve nearly identical Precision@5, the result demonstrates that **the embedding representation has a larger influence on retrieval quality than the choice between these two exact-search configurations.**

12. Final Conclusion

The experiment evaluated semantic retrieval using 20 ground-truth queries and top-5 retrieval from FAISS and ChromaDB. Precision@5 was calculated for every query and averaged across the complete test set. Both systems used the same Sentence Transformer embeddings to ensure a fair comparison. FAISS provides an efficient exact vector-search baseline, while ChromaDB combines vector search with database-oriented storage and filtering capabilities. If both systems obtain similar Precision@5, they provide comparable retrieval quality on this dataset; the preferred system can then be selected based on application requirements such as search speed, storage, metadata handling, and scalability.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Database Implementation	30%	Correctly builds and queries both FAISS and ChromaDB with accurate understanding of indexing	Builds both with minor issues	Only one database is built and queried correctly	Neither database functions correctly
Retrieval Relevance & Ranking	25%	Retrieval results are relevant and correctly ranked for all test queries	Mostly relevant results across queries	Inconsistent relevance across queries	Results are largely irrelevant
Comparative Analysis	20%	Thoughtful comparison of speed/accuracy trade-offs between the two databases	Reasonable comparison of the two databases	Superficial comparison with little insight	No meaningful comparison attempted
Benchmark Presentation	15%	Clear benchmark results presented (e.g. table or chart)	Results presented adequately	Results presented unclearly	No clear results presented
Methodology Documentation	10%	Clearly documents methodology and test queries used	Adequate documentation of methodology	Minimal documentation	No documentation of methodology

Marks Evaluation:

Question No.	Pipeline Functionality(30%)	Answer Relevance & Groundedness (25%)	Query Robustness(20%)	End-to-End Demo(15%)	Architecture Explanation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						